

AegisAI: An Agentic AI Policy Enforcement Gateway for Customer-Support Agents

PHASE I REPORT

Submitted By

HEMASHRI U 230701114

ISHITHA S 230701117

ISHWARI RAJMOHAN 230701118

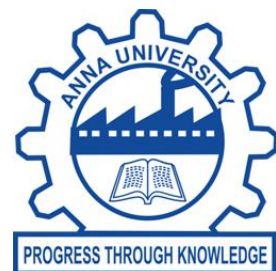
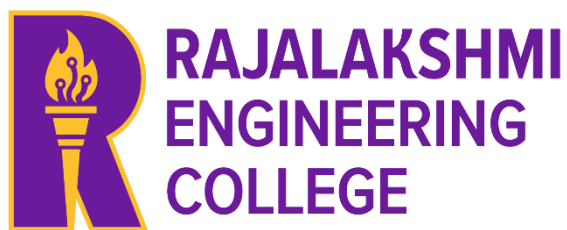
JAYA BHARATHI M 230701126

*In partial fulfilment for the award of the degree
of*

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY, CHENNAI

OCTOBER – 2026

BONAFIDE CERTIFICATE

Certified that this Report titled "**AEGISAI: An Agentic AI Policy Enforcement Gateway for Customer-Support Agents**" is the bonafide work of **Hemashri U (230701114)**, **Ishitha S (230701117)**, **Ishwari Rajmohan (230701118)**, **Jaya Bharathi M (230701126)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates.

“This project addresses the following Sustainable Development Goal(s):
SDG 9: Industry, Innovation and Infrastructure
SDG 16: Peace, Justice, and Strong Institutions”

SIGNATURE

Dr. J. Manoranjini, M.E., Ph.D.,
Professor and Head,
Department of Computer Science and
Engineering,
Rajalakshmi Engineering College,
Chennai-602 105.

SIGNATURE

Mrs. M. Divya, M.E.,
Assistant Professor,
Department of Computer Science and
Engineering,
Rajalakshmi Engineering College,
Chennai-602 105.

Submitted to Project Viva-Voce Examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

Enterprises globally are deploying customer support AI agents to automate replies and tool actions such as refunds, cancellations, and data access using probabilistic language models. However, these systems may also pose business risks by promising unauthorized refunds, leaking sensitive data, or performing unsafe actions. Most existing guardrails focus on content safety and post-hoc monitoring rather than preventing unsafe actions from reaching customers in the first place.

We present **AegisAI**, an agentic AI policy enforcement system with a multi-agent architecture that enforces customer-service policies before responding or executing business tools. It places a Policy Enforcement Gateway between a support agent and an operations agent, evaluates suggested actions according to deterministic business rules, and produces one of four decisions: allow, modify, block, or escalate. The system provides a strict authorization boundary that checks tool names and their arguments before action execution and maintains a unified audit trail from customer input to the final result.

On a 40-case benchmark covering refund limits, identity verification, delivery promises, PII protection, and escalation, AegisAI made the expected decision in all 40 cases with no false allows or false blocks, at an added latency of about 2.1 s per request, and recorded every decision in the audit trail. Our findings show that pre-action enforcement with inter-agent control is an executable and auditable approach for AI compliance in customer support.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavour to put forth this report. Our sincere thanks to our Chairman **Mr. S. MEGANATHAN, B.E, F.I.E.,** our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.,** and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.,** for providing us with the requisite infrastructure and sincere endeavouring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.,** our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. J. MANORANJINI, Ph.D.,** Professor and Head of the Department of Computer Science and Engineering for his guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **Mrs. M. DIVYA, M.E.,** Assistant Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project. We thank our Project Coordinator, **Dr. S. MADHUSUDHAN, Ph.D.,** Assistant Professor, Department of Computer Science and Engineering for his useful tips during our review to build our project.

HEMASHRI U	230701114
ISHITHA S	230701117
ISHWARI RAJMOHAN	230701118
JAYA BHARATHI M	230701126

TABLE OF CONTENT

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	05
	LIST OF TABLES	07
	LIST OF FIGURES	08
	LIST OF ABBREVIATIONS	09
1.	INTRODUCTION	11
	1.1 GENERAL	11
	1.2 OBJECTIVE	12
	1.3 EXISTING SYSTEM	12
	1.4 PROPOSED SYSTEM	14
2.	LITERATURE SURVEY	16
	2.1 LITERATURE REVIEW	16
	2.2 COMPARISON AND DISCUSSION	18
	2.3 CONCLUSION	19
3.	SYSTEM DESIGN	20
	3.1 SYSTEM ARCHITECTURE	20
	3.1.1 ARCHITECTURE FLOW	20
	3.1.2 COMPONENTS	21
	3.1.3 DATA DESIGN AND INTERFACES	22
	3.2 SYSTEM REQUIREMENTS	22
	3.2.1 HARDWARE REQUIREMENTS	22
	3.2.2 SOFTWARE REQUIREMENTS	23
	3.3 SYSTEM DESIGN	24
	3.3.1 ACTIVITY DIAGRAM	24
	3.3.2 DATA FLOW DIAGRAM	25
4.	PROJECT DESCRIPTION	29
	4.1 METHOLOGIES	29
	4.1.2 DESIGN PRINCIPLES APPLIED	29

4.1.3	DECISION METHODOLOGY	29
4.1.4	EVALUATION METHODOLOGY	30
4.2	CUSTOMER INTERACTION INTAKE AND INPUT GUARD	30
4.2.1	ORDER-CONTEXT RESOLUTION AT INTAKE	30
4.3	CUSTOMER-SUPPORT AGENT (AGENT A)	31
4.4	AEGISAI POLICY ENFORCEMENT GATEWAY	32
4.5	POLICY MANAGEMENT MODULE	33
4.6	MULTI-TURN CONTEXT STORE	34
4.7	OPERATIONS AGENT AND TOOL INTERCEPTOR	34
4.8	REMEDiation AGENT	35
4.9	AUDIT AND INCIDENT STORE	35
4.10	DASHBOARD AND ANALYTICS MODULE	36
5.	IMPLEMENTATION AND RESULT DISCUSSION	37
5.1	IMPLEMENTATION OVERVIEW	37
5.2	DEVELOPMENT ENVIRONMENT AND SETUP	37
5.3	IMPLEMENTATION OF THE POLICY ENFORCEMENT GATEWAY	38
5.4	IMPLEMENTATION OF POLICY DEFINITIONS	39
5.5	IMPLEMENTATION OF THE TOOL INTERCEPTOR AND AUTHORIZATION ENVELOPE	40
5.6	IMPLEMENTATION OF AUDIT LOGGING AND INCIDENT HANDLING	41
5.7	IMPLEMENTATION OF THE DASHBOARD	41
5.8	TESTING	42
6.	CONCLUSION AND FUTURE WORK	46
6.1	CONCLUSION	46
6.2	FUTURE WORK	47
	REFERENCES	48

LIST OF TABLES

TABLE NO.	TABLE TITLE	PAGE NO.
1.1	Summary of related systems and AegisAI	13
2.1	Comparison of related enforcement systems	18
3.1	REST endpoints	22
3.2	Hardware requirements	23
3.3	Software requirements	23
3.4	Level 1 Description of processes	27
3.5	Data stores	28
4.1	Design principles	29
4.2	Decisions and routes	29
4.3	Order resolution behaviour	31
4.4	Built-in policies	32
4.5	Demonstration scenarios	33
4.6	Policy management endpoints	34
4.7	Audit, incident and approval endpoints	35
5.1	Test suites	43
5.2	Decision accuracy	43
5.3	Latency	44
5.4	Benchmark results by policy category (N = 40)	44

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	System Architecture Diagram	20
3.2	Activity Diagram	25
3.3	Level 0 data flow diagram (context diagram)	26
3.4	Level 1 data flow diagram	27
4.1	Authorization envelope and interception	34
5.1	Project folder structure of AegisAI	37
5.2	Policy evaluation and decision aggregation code	39
5.3	Tool Interceptor halting a call with altered arguments	41
5.4	Conversation Simulator screen	42
5.5	Audit History, Incident Review and Policy Management screens	42
5.6	Automated test results	43

LIST OF ABBREVIATION

Abbreviation	Description
AI	Artificial Intelligence
API	Application Programming Interface
DFD	Data Flow Diagram
HITL	Human-in-the-Loop
JSON	JavaScript Object Notation
LangGraph	Language Graph Orchestration Framework
LLM	Large Language Model
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
ML	Machine Learning
MSE	Mean Square Error
NLP	Natural Language Processing
Ollama	Open Large Language Model Assistant
PII	Personally Identifiable Information
PostgreSQL	Postgres Structured Query Language
Qwen3-8B	Qwen Version 3 with 8 Billion Parameters
RMSE	Root Mean Square Error
REST	Representational State Transfer
TPU	Tensor Processing Unit
UI	User Interface
UML	Unified Modeling Language

Abbreviation	Description
YAML	Yet Another Markup Language
P50	50th Percentile / Median
P90	90th Percentile
P95	95th Percentile

CHAPTER 1

INTRODUCTION

1.1 General

Customer support is one of the first business functions to adopt large language model (LLM) agents. A support chatbot can answer questions about orders and delivery, explain return and refund rules and, when it is connected to business tools, act on a request by checking an order or issuing a refund. Because such systems respond instantly and handle many conversations at once, organizations use them to reduce the load on human support staff.

Autonomous support agents introduce unique risks because language models cannot reliably follow business policies, verification rules, or legal obligations. As Gupta et al. [1] highlight, agentic systems face security vulnerabilities that exploit their decision-making loops, leaving companies fully liable for unauthorized statements or actions, as demonstrated by three public incidents:

- **Moffatt v. Air Canada:** Air Canada was held liable for negligent misrepresentation after its chatbot falsely promised a customer a retroactive bereavement discount [2].
- **Chevrolet Dealership Chatbot:** A user tricked a dealership's AI chatbot into agreeing to sell a 2024 Chevrolet Tahoe for \$1 as a binding offer [3].
- **NYC MyCity Chatbot:** NYC's small-business chatbot gave illegal advice and wrong policy information, but the city kept it online anyway [4].

In each case the chatbot's output reached the user, which suggests that no independent check on policy compliance stood between the model and the user.

Existing safeguards filter harmful text or jailbreaks [5]–[8], but fail on business policies. A statement like "I can process your ₹3,000 refund" contains no toxic language, yet violates a ₹500 manager-approval limit. Post-event audits flag these mistakes too late to undo promises or transactions. Protecting business operations requires a pre-execution gateway that verifies proposed actions before they take effect.

This project presents **AegisAI**, a customer-service AI policy enforcement gateway built for that purpose. AegisAI sits between a support chatbot and the business tools it uses, and decides whether each proposal is allowed, modified, blocked, or escalated to a human. It is demonstrated on ShopSphere, a mock e-commerce store.

1.2 Objective

The aim of this project is to design, implement and evaluate a policy enforcement gateway that checks the responses and proposed business actions of an AI customer-support agent against company policy before they reach the customer or a business tool. The specific objectives are:

- **Separate Proposal from Execution:** Agent A proposes actions while Agent B executes only approved tasks.
- **Evaluate via Deterministic Policies:** Check proposals against YAML rules to return ALLOW, MODIFY, BLOCK, or ESCALATE decisions.
- **Authorize Exact Actions:** Issue authorization envelopes and verify tool names and arguments right before execution.
- **Leverage Conversation History:** Base policy decisions on multi-turn context like repeated complaints.
- **Sanitize and Escalate:** Replace unsafe replies with safe alternatives and forward high-risk cases to human reviewers.
- **Maintain Audit Trails:** Keep full incident logs and provide an API and dashboard for policy management.
- **Evaluate Performance:** Test on the ShopSphere benchmark to measure accuracy, false allows, false blocks, and latency.

The system is evaluated in a mock e-commerce environment with a locally hosted model. It is not deployed with real customers.

1.3 Existing System

Existing tools and research related tools and research fall into three groups. The descriptions below follow each system's own paper or documentation.

1.3.1 Guardrail toolkits and managed services

NeMo Guardrails [5] is an open-source tool used to add rules or “rails” to AI chatbot applications. Developers can define rules using Colang, which helps control how the AI behaves. Llama Guard [6] is an AI-based safety tool that checks both user inputs and AI responses. It uses a Llama 2-7B model to identify different types of safety risks based on a predefined set of categories. Guardrails AI [7] is an open-source Python framework that allows developers to add different validators to AI inputs and outputs. It can also check whether structured AI outputs follow a required format using Pydantic models. Amazon

Bedrock Guardrails [8] is a managed service that provides several safety controls. It can block unwanted topics, detect or hide sensitive information such as personal data, check whether responses are supported by the given context, and apply logical rules to validate AI responses.

1.3.2 Runtime enforcement for LLM agents

AgentSpec [9] is a domain-specific language for specifying runtime constraints on LLM agents through rules made of triggers, predicates and enforcement mechanisms; it was implemented for code execution, embodied agents and autonomous driving. VIGIL [10] checks an agent’s execution trace against behavioural policies, using a policy language that covers temporal dependencies, argument constraints and value-flow conditions, and reports over 95% recall with a false-positive rate below 10% on real agent runs. Agent Behavioral Contracts [11] bring Design-by-Contract principles to agents and are implemented in the AgentAssert runtime library. Policy-First Tooling [12] is a model-agnostic permission layer that mediates tool invocation through explicit constraints, risk-aware gating and auditable explanations; its authors report that stricter policy packs raised violation prevention from 0.000 to 0.681 while task success fell from 0.356 to 0.067. Mondl et al. [13] translate agent prompts, tool descriptions and policy documents into Cedar policies using an LLM generator–critic loop, and evaluate them on a medical benchmark.

1.3.3 Governance and security architectures

Governance-as-a-Service [14] is a modular, policy-driven enforcement layer that regulates agent outputs at runtime without changing model internals or requiring the agent’s cooperation; it uses declarative rules and a trust score, and logs every intercepted action. Tran et al. [15] argue that agent security should be treated as a networking problem, with enforcement placed outside the agent. Tallam [16] formalizes authorization propagation, the problem of keeping authorization invariants as agents delegate tasks and combine results, and derives seven structural requirements for authorization architectures. Habler et al. [17] analyse the security of Google’s Agent2Agent protocol and recommend secure development practices.

Table 1.1: Summary of related systems and AegisAI

SYSTEM	TYPE	WHAT IT CHECKS / ENFORCES
NeMo Guardrails [5]	Toolkit	Programmable rails on conversations and topics; protection against jailbreaks and prompt injection
Llama Guard [6]	Classifier model	Safety risks in prompts and responses, by taxonomy

SYSTEM	TYPE	WHAT IT CHECKS / ENFORCES
Guardrails AI [7]	Framework	Validators on LLM inputs and outputs; structured-output validation
Bedrock Guardrails [8]	Managed service	Denied topics, sensitive-information filters, grounding checks, Automated Reasoning checks on responses
AgentSpec [9]	Language and runtime	Rules with triggers, predicates and enforcement for agent actions
VIGIL [10]	Runtime enforcement	Agent execution traces against behavioural policies (temporal, argument and value-flow constraints)
Policy-First Tooling [12]	Permission layer	Tool invocation through constraints, risk-aware gating and auditable explanations
Governance-as-a-Service [14]	Enforcement layer	Agent outputs against declarative rules, with trust scores and logging
AegisAI (our project)	Pre-execution gateway	Customer-service policies on responses and actions; exact tool-argument check; multi-turn context; response remediation; unified audit trail

1.3.4 Limitations of existing systems

Four limitations stand out in the sources reviewed.

- **Lack of Built-In Business Rules:** Existing guardrail toolkits [5]–[8] offer general content filtering rather than domain-specific customer service rules, validating model responses instead of specific tool-call arguments.
- **Domain Mismatch:** Agent-enforcement research focus on coding, robotics, office tasks, and medical benchmarks [9], [10], [13] rather than customer-service operations.
- **Usability Trade-Offs:** Strict enforcement severely reduces task completion (e.g., dropping success rates from 35.6% to 6.7% [1.1]), requiring systems to provide helpful alternatives instead of flat blocks.
- **Generic Governance Frameworks:** Existing security architectures [14]–[17] define high-level authorization protocols rather than concrete business rules like refund limits.

Taken together, the sources we reviewed did not show a single system that combines pre-execution checking of customer-service policies, exact verification of tool-call arguments, multi-turn context, safe response remediation and a unified audit trail. This is the gap that AegisAI addresses.

1.4 Proposed System

AegisAI is a multi-agent policy enforcement gateway built on one principle: the model may propose, but only the gateway may authorize. A customer message first passes an Input

Guard that checks for fraud, legal threats and restricted topics. Agent A, a chatbot running Qwen3-8B through Ollama, returns a structured JSON object containing a reply and an optional action proposal. The Policy Enforcement Gateway, built with FastAPI, evaluates the proposal against policies defined in YAML and returns one of four decisions, resolved in the priority order ESCALATE, BLOCK, MODIFY, ALLOW:

- **ALLOW:** an authorization envelope naming the exact tool and arguments is created, and the action goes to Agent B.
- **MODIFY:** the response text is rewritten and evaluated again.
- **BLOCK:** tool execution is blocked by default and a safe alternative response is produced.
- **ESCALATE:** the conversation goes to a human-review queue with its context preserved.

Agent B, a restricted operations agent, receives only the authorization envelope. Before any tool runs, the Tool Interceptor compares the call with the envelope using exact equality on the tool name and every argument, without type coercion, and any mismatch halts execution. If an unhandled error occurs inside the gateway, the default verdict is BLOCK.

Every decision is stored in a PostgreSQL audit store with its request, policy, decision, evidence and tool-execution fields, and BLOCK and ESCALATE decisions also create incident records for human review. The same database holds conversation history, so that policies can use multi-turn context. A remediation module produces a compliant replacement reply using templates and LLM paraphrasing. A REST API and a React dashboard let staff review incidents and policy configurations, and the API supports manager approval or rejection of held actions. LangGraph orchestrates the workflow, and Docker Compose packages the system.

For example, when Agent A proposes a refund of ₹3,000, REFUND_LIMIT_001 returns BLOCK because no manager approval is present. Agent B is not called, the refund tool does not run, the customer receives a safe reply saying that the refund needs approval from the support team, and an incident is recorded.

Compared with the systems reviewed in Section 1.3, AegisAI is aimed specifically at customer-service policy. It checks actions before they run rather than only filtering text, verifies exact tool arguments, uses conversation history, produces a compliant replacement response instead of only blocking, and records every decision in one audit trail.

CHAPTER 2

LITERATURE SURVEY

2.1 Literature Review

This literature review examines key studies on agentic AI security [1], real-world chatbot failures [2]–[4], and thirteen technical papers [5]–[17]. As Gupta et al. [1] explain, goal-driven AI agents carry higher risks than simple reactive models because attackers can manipulate their decision-making process. The existing technical solutions are organized based on where they step in to protect the system.

2.1.1 Guardrail toolkits and managed services

NeMo Guardrails [5] adds programmable rails to LLM applications; the rails are user-defined, independent of the underlying model and interpretable. The authors of AgentSpec describe it as applying natural-language constraints at the dialogue level [9]. Llama Guard [6] is a Llama 2-7B model instruction-tuned to classify safety risks in prompts and responses against a taxonomy; it matches or exceeds existing moderation tools on the OpenAI Moderation Evaluation dataset and ToxicChat. Guardrails AI [7] builds input and output guards from shared validators and also validates structured output against Pydantic models. Amazon Bedrock Guardrails [8] offers denied topics, sensitive-information filters that block or mask personal data, contextual grounding checks, and Automated Reasoning checks that validate responses against rules the user defines.

2.1.2 Runtime enforcement for LLM agents

AgentSpec [9] is a domain-specific language whose rules have a trigger, a check made of predicates, and an enforcement action (user inspection, LLM self-examination, a predefined action, or stop). It evaluates each proposed agent action before it runs. The authors report that it prevented unsafe executions in over 90% of code-agent cases, eliminated all hazardous actions in the embodied-agent tasks and enforced 100% compliance in driving scenarios, at millisecond overhead. Success on safe embodied tasks fell from 58.62% to 54.26%. LLM-generated rules were less reliable, with 70.96% recall for embodied agents. The authors note that enforcement is at discrete checkpoints and does not reason about the long-term effect of an action sequence.

VIGIL [10] argues that action-level monitors miss violations that emerge across several individually permitted calls. It turns a run into a trace of typed events, compiles natural-language skill specifications into policies with temporal, argument and value-flow

conditions, and checks each pending call with an SMT solver. On 152 labelled runs (72 violating, 80 benign) it reached 95.8% recall at 89.6% precision, against F1 scores of 57.6% for AgentSpec, 50.3% for Progent and 76.8% for an LLM judge, versus 92.6% for VIGIL. The check takes 0.27 s, but one LLM call to compile the specification raises the average to 4.38 s per trace. The authors attribute its errors to steps hidden inside opaque scripts and to compilation errors, such as a generated rule that dropped the qualifier “bulk”.

Agent Behavioral Contracts [11] specify preconditions, invariants, governance policies and recovery mechanisms, enforced by the AgentAssert library. On AgentContract-Bench (200 scenarios, seven models, 1,980 sessions) contracted agents reached 88–100% hard-constraint compliance at under 10 ms per action; recovery succeeded in 100% of cases for frontier models but 17–100% across all models. Policy-First Tooling [12] places an enforcement point in front of tool calls, and a decision point returns allow, deny, require approval or transform with a logged rationale. In 225 trace-replay runs with simulated tools, schema checks and coarse allow/deny rules prevented no violations (0.000); argument constraints and budgets raised prevention to 0.597, and approval gates with redaction to 0.681, while task success fell from 0.356 to 0.067. Its authors note that policies may overfit the tool set and that the layer is not a full sandbox. Mondl et al. [13] generate Cedar policies from agent prompts, tool descriptions and policy documents through a generator–critic loop; on MedAgentBench the policies covered substantially more of the written specification than hand-coded enforcement.

2.1.3 Governance and security architectures

Governance-as-a-Service [14] interposes a layer between untrusted agents and their environment. Declarative JSON rules are checked by deterministic pattern matching, each action gets an allow, warn, block or escalate decision, and a trust factor built from violation history drives escalation. In essay-writing and trading simulations with three open-source models, it blocked 33 of 42 trades in the governed run and 693 risky trades in the adversarial run. The authors’ own comparison table lists latency benchmarks as still needed. Tran et al. [15] argue that agent security should be treated as a networking problem, with enforcement placed outside the agent. Tallam [16] names the problem of authorization propagation in multi-agent systems, identifies three sub-problems (transitive delegation, aggregation inference, temporal validity) and derives seven structural requirements for authorization architectures. Habler et al. [17] threat-model Google’s Agent2Agent protocol, covering Agent Card management, task-execution integrity and authentication.

2.2 Comparison and Discussion

Table 2.1 compares the works that report an enforcement mechanism with an evaluation, using only what their sources state.

Table 2.1: Comparison of related enforcement systems

WORK	MECHANISM	EVALUATION SETTING	REPORTED RESULT
Llama Guard [6]	LLM classifier of prompt and response safety risks	OpenAI Moderation dataset, ToxicChat	Matches or exceeds existing moderation tools
AgentSpec [9]	Rules (trigger, check, enforce) before agent actions	Code, embodied and driving agents	>90% unsafe code prevented; ms overhead
VIGIL [10]	Whole-trace policies checked by SMT	152 labelled runs	95.8% recall, 89.6% precision; 4.38 s per trace
Behavioral Contracts [11]	Preconditions, invariants, governance, recovery	AgentContract-Bench, 1,980 sessions	88–100% hard-constraint compliance; <10 ms
Policy-First Tooling [12]	Allow, deny, approve, transform at tool calls	225 trace-replay runs	Prevention 0.000 to 0.681; success 0.356 to 0.067
Mondl et al. [13]	LLM generator–critic writes Cedar policies	MedAgentBench	More of written policy covered than hand-coded rules
GaaS [14]	JSON rules; allow, warn, block, escalate; trust factor	Essay and trading simulations	33 of 42 trades blocked; 693 in adversarial run
AegisAI (our project)	Pre-execution gateway: YAML policies; four decisions; exact tool-argument check; multi-turn context; remediation; audit	ShopSphere mock store, 40 cases	40/40 expected decisions; +2.1 s mean latency

Four key observations emerge from the literature:

- **Different Intervention Points:** Existing systems intervene at varying stages of execution. Text-based toolkits and classifiers [5]–[8] filter input and output words, frameworks like

AgentSpec [9], Policy-First Tooling [12], and GaaS [14] check individual actions or outputs, and full-trace monitors like VIGIL [10] evaluate the entire conversation history.

- **Usability Trade-Offs:** Stricter safety enforcement often reduces system usability [9], [12]. Instead of flatly blocking requests, AegisAI handles interventions gracefully by generating safe, compliant replacement responses on MODIFY and BLOCK decisions, while routing high-risk edge cases to human review.
- **Latency and Policy Predictability:** Safety checks introduce varying delays, ranging from a few milliseconds in lightweight models [9], [11] to 4.38 seconds in deep LLM trace evaluations [10]. AegisAI averages 2,127.96 ms through its gateway (compared to 4.44 ms for the Agent A draft stage alone). Additionally, while AI-generated rules [9] and complex compilation [10] introduce errors, AegisAI relies on hand-written YAML policies that evaluate deterministically to guarantee predictable behavior.
- **Lack of Customer Service Focus:** Existing benchmarks focus on coding, driving, office tasks, medical checks, and financial simulations [9], [10], [12]–[14]. While features like audit trails [12], [14], context-aware decisions [10], [14], and action transformation [11], [12] exist across separate systems, no single existing model combines customer-service business rules, strict tool-argument verification against authorization limits, and multi-turn context tracking.

2.3 Conclusion

The literature moves from filtering model text [5]–[8] towards enforcing rules on agent actions and execution traces [9]–[14], supported by work on governance, delegation and protocol security [15]–[17]. Current findings show that external policies can effectively block unsafe actions, though stricter enforcement lowers task completion rates [9], [12]. Additionally, safety processing times vary from milliseconds to seconds based on how heavily they rely on model reasoning [9]–[11]. Crucially, existing research lacks evaluation within customer-service policy settings. AegisAI fills this exact gap by providing a pre-execution gateway that unifies business policy enforcement, precise tool-argument verification, multi-turn context tracking, safe response remediation, and an integrated audit trail.

CHAPTER 3

SYSTEM DESIGN

3.1 System Architecture

AegisAI is organised as a layered system (Figure 3.1). Customer messages arrive from the ShopSphere chat interface, which acts only as an external client, and pass through a FastAPI layer into a LangGraph workflow that contains the agents, the Policy Enforcement Gateway and the Tool Interceptor. Policies are defined in YAML, Agent A runs on a local Qwen3-8B model served by Ollama, and a PostgreSQL database stores conversations, policies, audit events, incidents and approvals. Docker Compose packages the services. The mock e-commerce tools sit behind the Tool Interceptor, so no business tool can be reached except through the gateway.

3.1.1 Architecture Flow

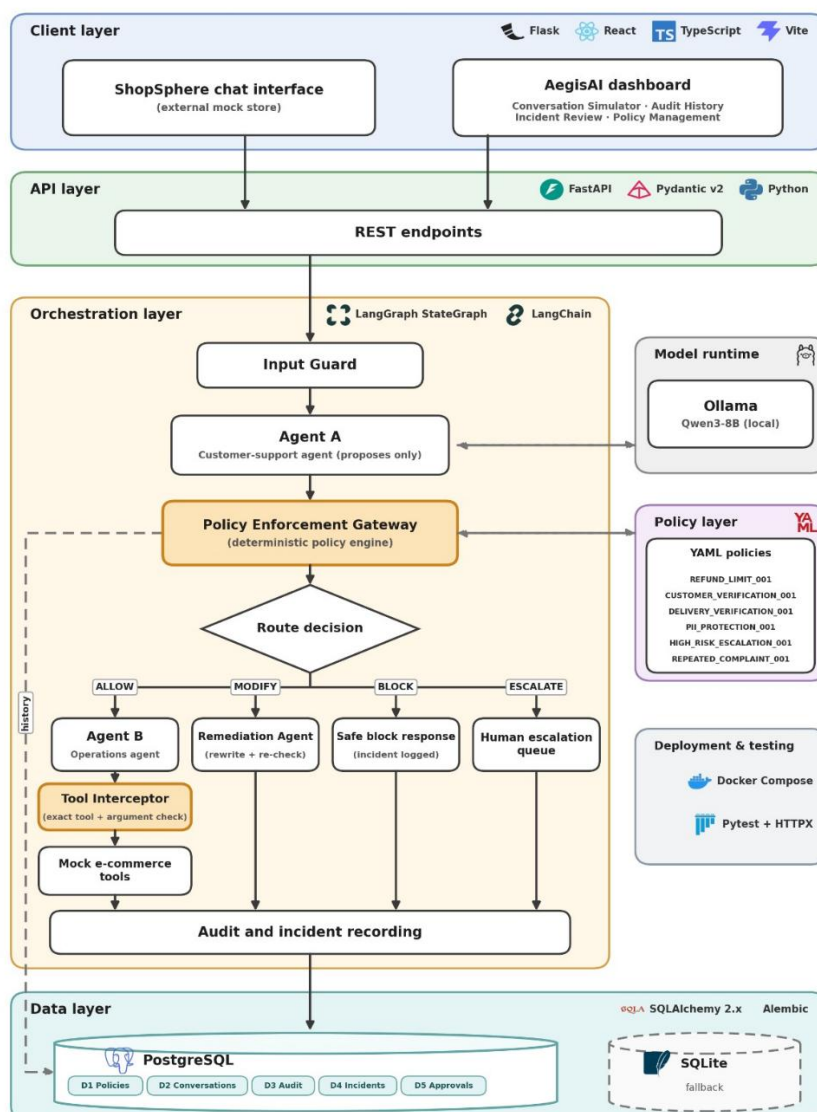


Figure 3.1: System Architecture Diagram

3.1.2 Components

Table 3.1 lists the components of Figure 3.1 and the responsibility of each, as described in the project documentation.

1. Input Guard. The customer message first passes through the Input Guard before it reaches the customer-support agent.

2. Agent A – Customer-Support Agent. Agent A is the customer-support chatbot, built on the Qwen3-8B model served through Ollama. It generates the customer-facing response and may propose a business action. Agent A is *proposal-only*: it can express an action only as structured JSON and has no direct permission to run any tool.

3. AegisAI Gateway. The gateway is the core of the system. It is a deterministic engine that runs a priority pipeline over the proposed response and action and evaluates them against the registered policies. The outcome of this evaluation is one of ALLOW, MODIFY, BLOCK or ESCALATE.

4. Agent B – Operations Agent. Agent B is called only when the decision is ALLOW. It receives an *authorization envelope* containing only the allowed tool, its parameters, and the decision ID.

5. Tool Interceptor. The ToolInterceptor checks the tool name and every argument against the authorization envelope immediately before the tool is invoked. If any argument is missing or has been modified, execution is blocked.

6. Remediation Agent. When the decision is MODIFY, the Remediation Agent repairs the response. In the current status of the project, response remediation is partially complete, with LLM-based rewriting still in progress.

7. Safe Response Generation. When the decision is BLOCK, a safe response is generated to replace the unsafe proposal.

8. Human Escalation Queue. When the decision is ESCALATE, a high-severity incident is queued for human review. Managers can approve or reject through the approval endpoints.

9. Audit Store. Every evaluation is recorded in the audit store, which is backed by PostgreSQL or SQLite. It holds audit events, incidents, policies and approvals.

The workflow is implemented as a LangGraph StateGraph. A shared state carries the request, conversation and customer identifiers, the customer message, Agent A's response,

the proposed action, the policy decision, the approved action, the tool result, the final response, the audit event and any error through the nodes `input_guard`, `support_agent`, `normalize_agent_output`, `policy_gateway`, `route_decision`, `operations_agent`, `tool_interceptor`, `execute_mock_tool`, `remediation_agent`, `audit_event`, `human_escalation` and `final_response`. Routing follows the decision: ALLOW goes to the operations agent, MODIFY to the remediation agent, BLOCK to the audit event and a safe response, and ESCALATE to human escalation and then the audit event. A blocked action is never executed.

3.1.3 Data design and interfaces

The database holds seven tables: `conversations`, `messages`, `policies`, `policy_versions`, `audit_events`, `incidents` and `approvals`. Each audit event records the `request_id`, `conversation_id`, `customer_id`, Agent A's response, the proposed action, the `policy_id` and `policy_version`, the decision, severity, evidence, whether a tool was executed, the safe response and a timestamp. An incident record is added for BLOCK and ESCALATE decisions and carries the conversation details and the policy details. Table 3.2 lists the REST endpoints exposed by the API layer.

Table 3.1: REST endpoints

METHOD	ENDPOINT	PURPOSE
POST	<code>/api/chat</code>	Run the full pipeline on a customer message
GET	<code>/api/audits</code> , <code>/api/audits/{event_id}</code>	Audit trail with filtering and pagination; full audit event
GET	<code>/api/incidents</code>	Incident queue
GET	<code>/api/policies</code>	Registered policies and active versions
POST, PUT	<code>/api/policies</code> , <code>/api/policies/{policy_id}</code>	Draft a new policy; create a new policy version
POST	<code>/api/approvals/{decision_id}/approve</code> , <code>.../reject</code>	Manager approval or rejection of a held action
GET	<code>/api/health</code>	Service health and active policy count

3.2 System Requirements

3.2.1 Hardware requirements

The only component with notable hardware demands is the language model, because Qwen3-8B runs locally through Ollama with no cloud dependency. The gateway, policy engine, database and dashboard are ordinary web-application components for which the

project documentation sets no minimum. Table 3.3 gives the requirements that can be stated from published figures for the model.

Table 3.2: Hardware requirements

RESOURCE	REQUIREMENT
GPU memory	About 5 GB of VRAM for the Q4_K_M build of Qwen3-8B
System memory	About 10 GB of RAM
Storage for the model	About 5.2 GB for qwen3:8b
Processor	No minimum documented; CPU-only inference is possible but slower than on a GPU
Network	Not required for model inference, since the model runs locally
Other services	PostgreSQL, the backend and the frontend run as Docker Compose services; no minimum documented

These figures are published estimates for running Qwen3-8B, not measurements of AegisAI. The automated tests use a deterministic mock model provider, so they do not need the model to be running.

3.2.2 Software requirements

Table 3.4 lists the software used to build and run AegisAI. Versions are given only where the project documents state them.

Table 3.3: Software requirements

CATEGORY	TECHNOLOGY	ROLE
Backend language	Python 3.11	Gateway, policy engine, Tool Interceptor, mock tools, audit logic
Web framework	FastAPI	REST API for chat, policies, audits, incidents and approvals
Data validation	Pydantic v2	Validates proposals, policy decisions, tool arguments and audit events
Orchestration	LangGraph; LangChain Ollama integration	Multi-agent workflow, conditional routing, model connector
Model runtime	Ollama with qwen3:8b	Local model for Agent A
Policy configuration	YAML	Human-readable policy definitions loaded at runtime
Database	PostgreSQL 16 (SQLite fallback)	Conversations, policies, audit events, incidents, approvals

CATEGORY	TECHNOLOGY	ROLE
ORM and migrations	SQLAlchemy 2.x; Alembic	Database access and schema changes
Frontend	React 18, TypeScript, Vite, Tailwind CSS, Recharts; Node.js with npm	Dashboard screens
Deployment	Docker; Docker Compose	Packaging and orchestration of the services
Testing	Pytest; HTTPX	Unit, workflow and API tests
Containerization	Docker, Docker Compose	Containerized deployment
Client	Web browser	Access to the dashboard
Operating system	Windows	Any system that supports Docker and Ollama

3.3 System Design

3.3.1 Activity diagram

Figure 3.2 shows the activities performed for one customer message. The message enters through the chat endpoint and the Input Guard performs its basic checks. Agent A then returns a reply and an optional action proposal, the output is normalized, and the Policy Enforcement Gateway evaluates every applicable policy using the proposal and the conversation context. The results are aggregated into a single verdict by the priority ESCALATE, BLOCK, MODIFY, ALLOW, and the workflow branches on it.

On ALLOW, an authorization envelope with the exact tool name and arguments is created and Agent B attempts the call; the Tool Interceptor executes the mock tool only if the call equals the envelope, and halts it otherwise. On MODIFY, the Remediation Agent rewrites the response and the revised text is evaluated again. On BLOCK, tool execution is blocked, a safe alternative response is generated and an incident is created. On ESCALATE, the case is routed to the human-review queue with its context preserved and an incident is created. An unhandled error inside the gateway also produces a BLOCK verdict. Every path ends with an audit event and the final response to the customer.

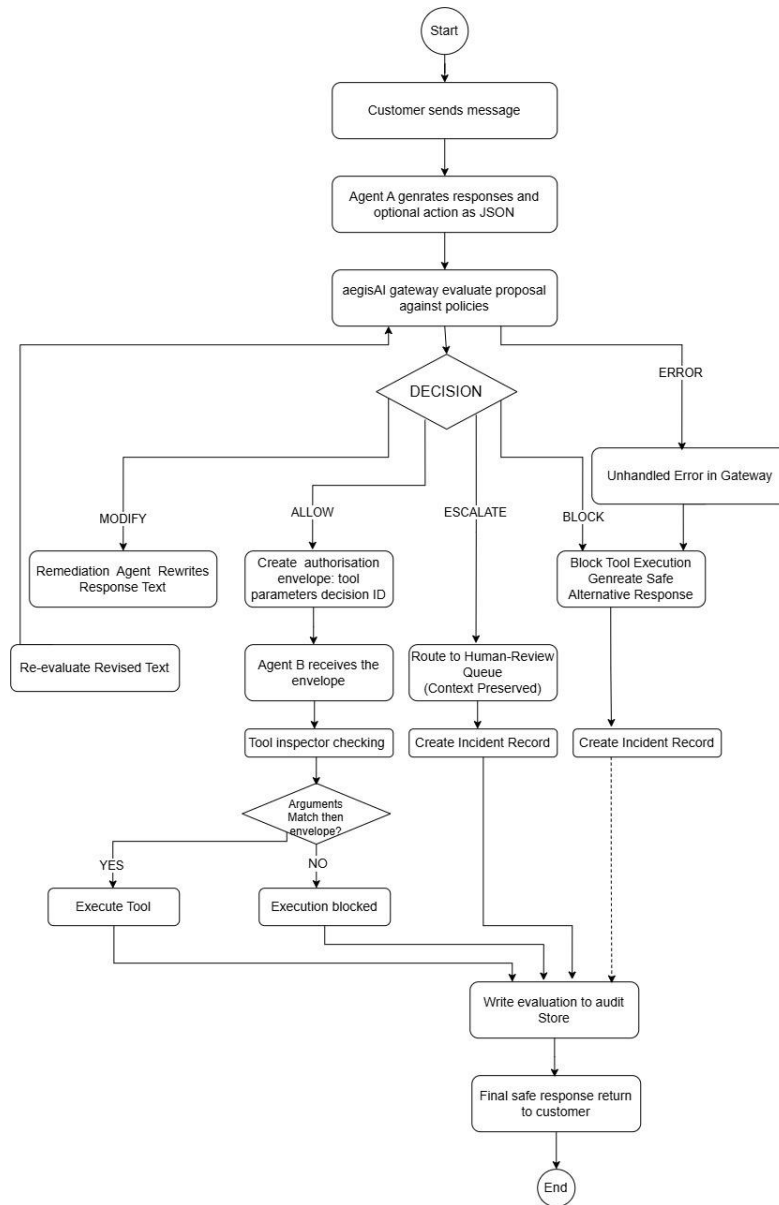


Figure 3.2: Activity diagram

3.3.2 Data flow diagram

The data flow diagrams use rectangles for external entities, ellipses for processes and open rectangles for data stores. Figure 3.3 is the context diagram. AegisAI appears as one process with three external entities: the customer, who sends messages and receives only the final policy-checked response; the compliance officer or manager, who changes policies, makes approval decisions and reviews incidents; and the mock e-commerce tools, which receive only authorized calls and return results. Three data stores hold the active policies (D1), the conversations (D2) and the audit events and incidents (D3).

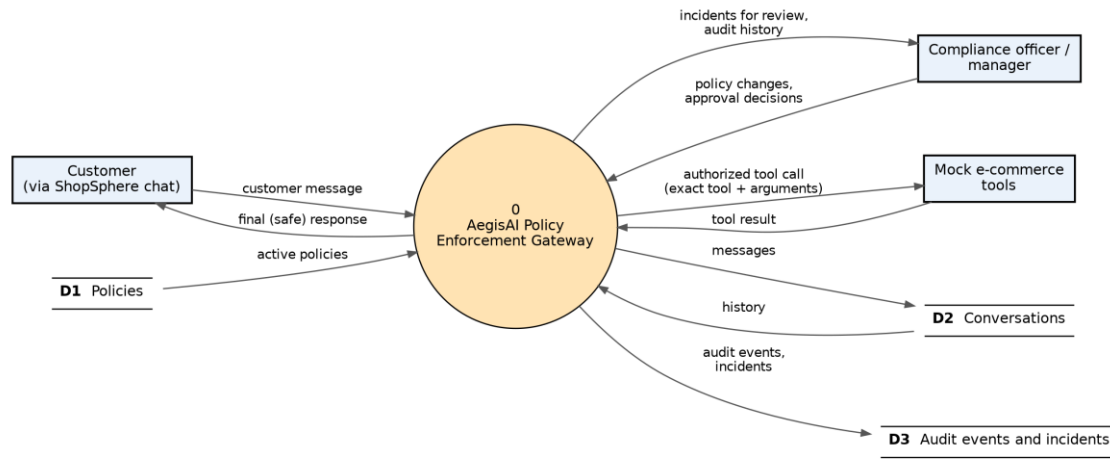


Figure 3.3: Level 0 data flow diagram (context diagram)

Figure 3.4 expands the process into eight sub-processes and five data stores. Process 1.0 (Input Guard) checks the customer message and stores it in D2. Process 2.0 (Agent A) produces the response and action proposal. Process 3.0 (Policy Gateway) reads the policies in D1 and the history in D2 and routes its result: an authorization envelope to 4.0 for ALLOW, violation details to 5.0 for MODIFY and BLOCK, and the case context to 7.0 for ESCALATE. Process 4.0 (Agent B and Tool Interceptor) sends the authorized call to the tools and returns the execution outcome. Process 5.0 (Remediation Agent) returns a safe response, and 7.0 (human review and approval) returns the review outcome and writes approval records to D5. Process 6.0 records the audit event in D3, writes an incident to D4 for BLOCK and ESCALATE decisions, and returns the final response to the customer. Process 8.0 lets the compliance officer manage policies, creating a new policy version in D1.

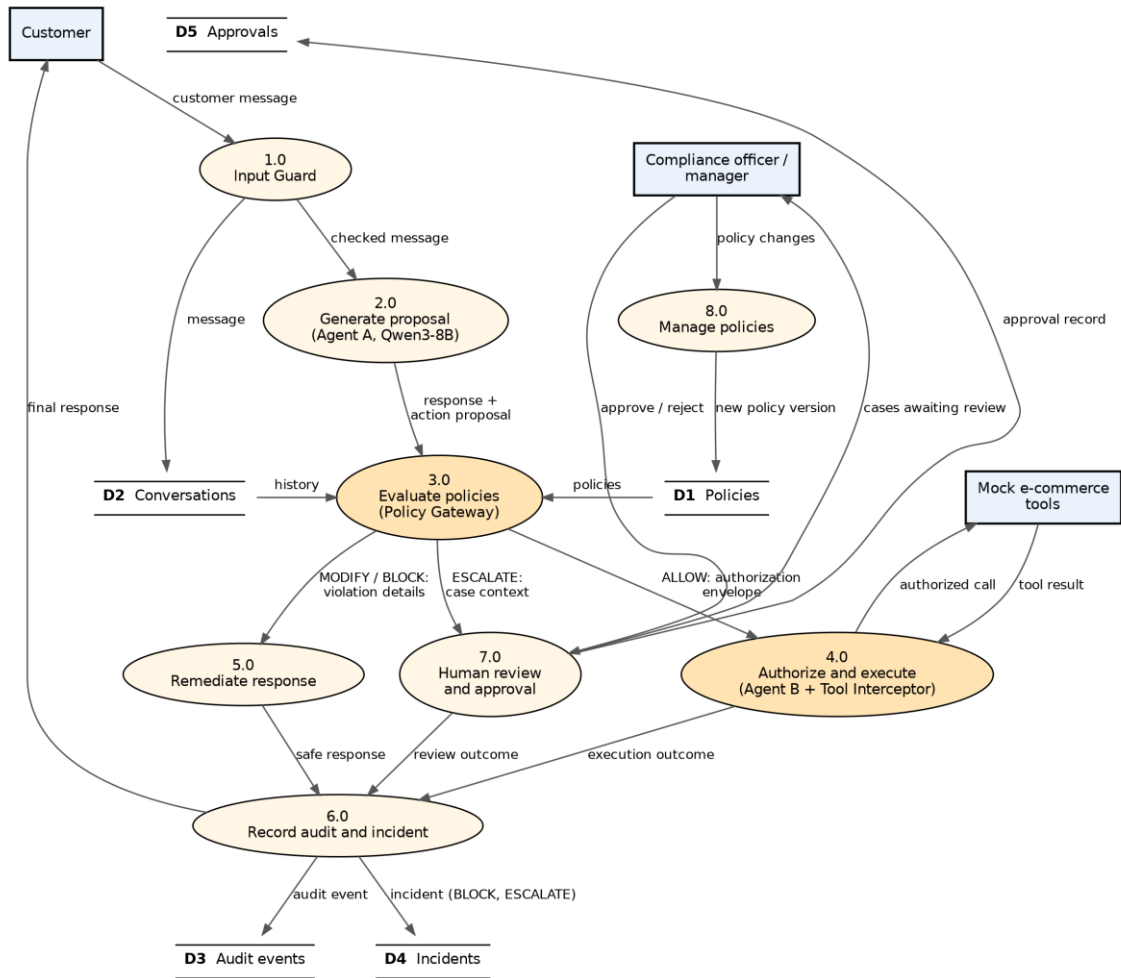


Figure 3.4: Level 1 data flow diagram

Table 3.4 – Level 1 Description of processes

PROCESS	DESCRIPTION	INPUTS	OUTPUTS
1.0 Input Guard	Checks the customer message before it reaches the model	Customer message	Checked message
2.0 Agent A: Generate Proposal	Qwen3-8B produces the response and an optional business action as structured JSON; it cannot run tools	Checked message	Proposed response and action
3.0 Gateway: Policy Evaluation	Deterministic priority pipeline evaluates the proposal against policies and issues the decision	Proposal, active policies	ALLOW / MODIFY / BLOCK / ESCALATE, decision record
4.0 Authorized Execution	Agent B receives the authorization envelope; the Tool Interceptor verifies tool name and arguments before invocation	Authorization envelope	Tool call, execution result, or a blocked call
5.0 Remediate Response	Remediation Agent repairs a response that violated policy	MODIFY decision, original response	Repaired response

PROCESS	DESCRIPTION	INPUTS	OUTPUTS
6.0 Generate Safe Response	Replaces an unsafe proposal with a safe response	BLOCK decision	Safe response
7.0 Escalate to Human Review	Queues a high-severity incident for review	ESCALATE decision	Incident in queue

Table 3.5 – Data stores

DATA STORE	CONTENT
D1 Policy Store	YAML policies loaded into the policy engine, with registered policies and active versions
D2 Conversations	Conversations and messages, plus per-conversation context: verification status, prior refunds, verified order lookups, complaint count, escalation status
D3 Audit Store	Audit events, incidents, policies and approvals, held in PostgreSQL or SQLite
D4 Incidents	Auto-created records for high-severity BLOCK and ESCALATE decisions: decision, policy ID and version, evidence, severity, review status
D5 Approvals	Single-use authorization envelopes issued on ALLOW (exact tool, arguments, policy ID and version) and human review outcomes for escalated cases

Key observations from the DFD

- Agent A never communicates with the business tools directly; every proposed action must pass through process 3.0.
- Only the ALLOW decision leads to Agent B and to the business tools, and only with an authorization envelope checked by the Tool Interceptor.
- Every path writes its result to the Audit Store, which reflects the audit-first principle.
- The human reviewer participates in two flows: reviewing escalated incidents and approvals, and managing policy versions.

CHAPTER 4

PROJECT DESCRIPTION

4.1 Methodologies

4.1.1 Overall Approach

The project follows a **pre-execution enforcement** approach. Most systems discover problems after the chatbot has already responded or acted. AegisAI instead detects a violation and then stops or repairs it before damage happens. To achieve this, the system separates the *proposing* of a response or action from the *deciding* and the *executing* of it:

The customer-support agent (Agent A) only **proposes** a response and an optional action.

A deterministic gateway **decides** whether the proposal complies with policy.

A separate operations agent (Agent B), guarded by a tool interceptor, **executes** only what has been authorized.

4.1.2 Design Principles Applied

Four principles guide every module in this chapter.

Table 4.1 – Design principles

PRINCIPLE	ROLE
Proposal-only	Agent A can only propose actions as structured JSON. It has no direct permission to run tools.
Authorization envelope	Agent B receives only the allowed tool, its parameters and the decision ID.
Pre-execution interception	The ToolInterceptor checks the tool name and every argument against the authorization envelope immediately before invocation. Missing or modified arguments block execution.
Fail-closed, audit-first	Every evaluation is recorded, and high-severity violations create reviewable incidents.

4.1.3 Decision Methodology

Each interaction ends in one of four decisions, which also determine the route taken through the system.

Table 4.2 – Decisions and routes

DECISION	ROUTE TAKEN
ALLOW	Agent B is called, the Tool Interceptor verifies the call, and the tool executes.

DECISION	ROUTE TAKEN
MODIFY	The Remediation Agent repairs the response.
BLOCK	A safe response is generated; the tool does not execute.
ESCALATE	The case goes to the human escalation queue.

The policy checks are carried out by a deterministic engine using a priority pipeline. Policies are defined in YAML and loaded into the policy engine at runtime, so the decision logic is rule-based and repeatable rather than dependent on the language model.

4.1.4 Evaluation Methodology

The system was evaluated end to end against the live AegisAI policy gateway, running at <http://127.0.0.1:8000>, together with the ShopSphere application. A benchmark of **40 cases** was used, divided equally into five policy categories of eight cases each: delivery-date, escalation, identity-verification, PII-protection and refund-limit. Results were recorded in a CSV file (`eval_results.csv`). The following measures were used:

- Overall decision accuracy
- False-allow rate and false-block rate
- Argument invariant block rate
- Latency, with and without the gateway (mean, median, P90 and P95)

In addition, the project uses four automated test suites, described in Section 5.8.1 and summarized with the measured results in Section 5.8.2.

4.2 CUSTOMER INTERACTION INTAKE AND INPUT GUARD

This module is the entry point of the system. A customer message is received through the POST `/api/chat` endpoint, which runs the full pipeline on a customer message. Before the message reaches the customer-support agent, it passes through the **Input Guard**. The Input Guard is the first stage of the architecture, so no customer message reaches Agent A without first being checked.

4.2.1 Order-Context Resolution at Intake

The project also handles messages that concern an order but do not name one, such as "Where is my order?", "Cancel my order" or "I want a refund". Instead of letting an order-related action be proposed with a missing order ID, the system looks up the customer's order history through OrderService and behaves as follows:

Table 4.3: Order resolution behaviour

SITUATION	BEHAVIOUR
Exactly one candidate order	The order ID is resolved to that order automatically.
Several candidate orders	The system asks a clarifying question listing the active order IDs.
No orders	The customer is asked to supply the order ID.

4.3 CUSTOMER-SUPPORT AGENT (AGENT A)

Agent A is the customer-support chatbot. It is built on the **Qwen3-8B** model served through **Ollama**, with workflow state managed by **LangGraph** and langchain-ollama. It generates the customer-facing response and, when needed, proposes a business action.

4.3.1 Proposal-Only Behaviour

The central rule for Agent A is that it is **proposal-only**. It can propose actions only as structured JSON, and it has no direct permission to run tools. For example, a refund proposal takes the form of a tool name and its arguments:

```
{
  "tool_name": "issue_refund",
  "arguments": { "order_id": "ORD-101", "amount": 3000 }
}
```

Because Agent A cannot execute anything itself, a mistaken or unsafe proposal cannot cause harm unless the gateway approves it.

4.3.2 Risks Addressed

Agent A is the component through which the following risks arise, and they are the reason the gateway exists:

- Promising a refund above the company's limit.
- Revealing another customer's order details or personal information.
- Promising a delivery date without checking the order system.
- Approving compensation, cancellations or account changes without authorization.
- Failing to escalate fraud complaints or legal threats.

4.3.3 Intent Handling for Commercial Discount Requests

During development, the intent classification feeding the agent was adjusted so that requests for discounts, concessions, loyalty benefits and promotions take precedence over catalog keywords that might appear in the same message (for example a product name such as "smart watch"). A custom commercial discount request on an order is mapped to supervisor

review and escalation, shown to the customer as "Supervisor Approval Required". Regression tests cover the original query and three paraphrased variants.

4.4 AEGISAI POLICY ENFORCEMENT GATEWAY

The gateway is the core of the system. It receives the response and optional action proposed by Agent A and evaluates them against the registered policies. It is a deterministic engine with a priority pipeline, and its output is one of ALLOW, MODIFY, BLOCK or ESCALATE.

4.4.1 Built-In Policies

Table 4.4: Built-in policies

POLICY ID	SEVERITY	RULE
REFUND_LIMIT_001	HIGH	Refunds above ₹500 require manager authorization.
CUSTOMER_VERIFICATION_001	HIGH	Customer identity must be verified before order details or financial operations.
DELIVERY_VERIFICATION_001	MEDIUM	No specific delivery-date promises unless confirmed by a verified order lookup.
PII_PROTECTION_001	CRITICAL	Card numbers, CVVs, passwords, full phone numbers and cross-customer data are blocked.
HIGH_RISK_ESCALATION_001	CRITICAL	Fraud complaints, legal threats and consumer-court mentions are escalated immediately.

A sixth policy, DUPLICATE_REFUND_001, is also present in the gateway code.

4.4.2 Structured Decision Record

Every evaluation returns a structured record. For the blocked ₹3,000 refund the record is:

```
{
  "decision": "BLOCK",
  "severity": "HIGH",
  "policy_id": "REFUND_LIMIT_001",
  "reason": "Refund amount of ₹3,000.00 exceeds automatic threshold of ₹500.00 and lacks manager authorization.",
  "proposed_action": {
    "tool_name": "issue_refund",
    "arguments": { "order_id": "ORD-101", "amount": 3000 }
  },
  "approved_action": null,
```

```

    "requires_human_review": true,
    "tool_executed": false,
    "agent_b_called": false
}

```

The record states the decision, the severity, the policy that fired, the reason, and whether the tool was executed and whether Agent B was called. This makes each decision explainable and traceable.

4.4.3 Demonstration Scenarios

Table 4.5: Demonstration scenarios

SCENARIO	DECISION
₹3,000 refund without manager approval	BLOCK (REFUND_LIMIT_001); Agent B not called, no tool run, incident created
₹400 refund for a verified customer	ALLOW; Agent B called with an authorization envelope, refund executes
Unverified access to order details	BLOCK (CUSTOMER_VERIFICATION_001)
Unverified delivery-date promise	MODIFY (DELIVERY_VERIFICATION_001); response rewritten to ask for a verified order ID
Fraud or legal threat	ESCALATE (HIGH_RISK_ESCALATION_001); high-severity incident queued
Tampered tool arguments	BLOCK; the interceptor halts the modified call

The gateway runs as a FastAPI service. The GET /api/health endpoint reports service health and the number of active policies.

4.5 POLICY MANAGEMENT MODULE

The policy management module controls how policies are stored, viewed and changed. Policies are defined in YAML and loaded into the policy engine at runtime. The database also holds the policies, with their versions.

As for Versioning a change to an existing policy is made by creating a **new policy version** rather than overwriting the old one, and the registered policies are listed together with their active versions. A new policy is first created as a draft. This supports review and traceability of rule changes.

Table 4.6: Policy management endpoints

METHOD	ENDPOINT	PURPOSE
GET	/api/policies	Registered policies and active versions
POST	/api/policies	Draft a new policy
PUT	/api/policies/{policy_id}	Create a new policy version

4.6 MULTI-TURN CONTEXT STORE

Some policy decisions depend on what happened earlier in a conversation, for example whether the customer's identity has already been verified, or whether an order lookup has already confirmed a delivery date. The multi-turn context store is meant to give the gateway this conversational memory so that checks are not limited to a single message.

4.7 OPERATIONS AGENT AND TOOL INTERCEPTOR

4.7.1 Operations Agent (Agent B)

Agent B is the operations agent. It is called only when the gateway decision is **ALLOW**. It does not receive the full proposal from Agent A; it receives an **authorization envelope** containing only:

the allowed tool, its parameters, and the decision ID.

4.7.2 Tool Interceptor

The ToolInterceptor is the last line of control before a tool runs. Immediately before invocation it checks the tool name and every argument against the authorization envelope. A missing or modified argument blocks execution.

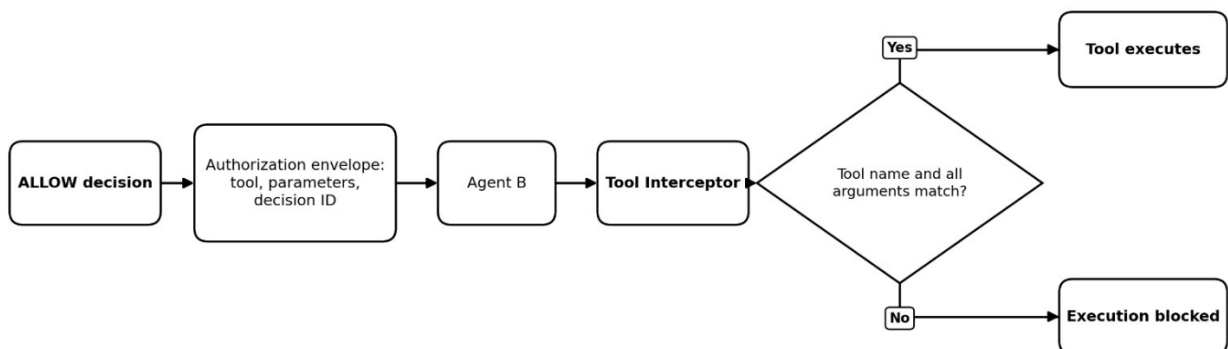


Figure 4.1 – Authorization envelope and interception

4.7.3 Security Value

The interceptor guards against a tool call that is changed between the gateway's approval and the actual invocation. In the demonstration scenarios, a tampered tool-argument case

ends in **BLOCK** because the interceptor halts the modified call. In the benchmark, the argument invariant block rate was 97.5% (Section 4.1.6).

4.8 REMEDIATION AGENT

The Remediation Agent handles the **MODIFY** decision. When a proposed response violates a policy but can be repaired, the agent rewrites it into a compliant form instead of discarding it completely.

4.8.1 Relationship with Safe Response Generation

The Remediation Agent repairs a response under **MODIFY**. For a **BLOCK** decision, a separate Safe Response Generation step produces the replacement response, as in the ₹3,000 refund example where the unsafe response is replaced with a safe one.

4.9 AUDIT AND INCIDENT STORE

The audit and incident store is where AegisAI records what happened. Every evaluation is written to it, following the **fail-closed, audit-first** principle, and high-severity violations create reviewable incidents. The store is implemented with **PostgreSQL or SQLite**, accessed through **SQLAlchemy 2.x** with **Alembic** for migrations. It holds audit events, incidents, policies and approvals.

Table 4.7: Audit, incident and approval endpoints

METHOD	ENDPOINT	PURPOSE
GET	/api/audits	Audit trail with filtering and pagination
GET	/api/audits/{event_id}	Full audit event
GET	/api/incidents	Security and policy incident queue
POST	/api/approvals/{decision_id}/approve	Manager approval
POST	/api/approvals/{decision_id}/reject	Manager rejection

4.9.1 Incidents and Human Review

An **ESCALATE** decision, such as a fraud complaint or a legal threat, creates a high-severity incident that is queued for human review. A **BLOCK** of a high-severity violation, as in the ₹3,000 refund, also creates an incident. A manager can then approve or reject a decision through the approval endpoints.

4.10 DASHBOARD AND ANALYTICS MODULE

The dashboard is the interface through which reviewers and managers see the behaviour of AegisAI. It is built with **React, TypeScript, Vite and Tailwind CSS**, and it reads its data from the backend endpoints for audits, incidents and policies. Using the data exposed by the API, the dashboard works with the audit trail, with filtering and pagination, full details of an individual audit event, the queue of security and policy incidents, and the registered policies and their active versions.

This chapter described the methodology and the modules of AegisAI. The system separates proposing, deciding and executing: Agent A proposes, the deterministic gateway decides with ALLOW, MODIFY, BLOCK or ESCALATE, and Agent B executes only what the authorization envelope permits, under the check of the Tool Interceptor. Policy management, the audit and incident store, and the dashboard provide control and visibility, while the multi-turn context store and the remediation agent are partially complete. On the 40-case benchmark the gateway reached 100% decision accuracy with no false allows or false blocks, at a mean added latency of about 2.12 seconds per interaction.

CHAPTER 5

IMPLEMENTATION AND RESULT DISCUSSION

5.1 Implementation Overview

AegisAI is implemented in Python and TypeScript as a set of cooperating modules that form the pre-execution enforcement pipeline described in Chapter 3. LangGraph connects the steps as a StateGraph, and a FastAPI layer exposes them to the dashboard. Table 5.1 maps each module to the technology used to implement it.

5.2 Development Environment and Setup



Figure 5.1: Project folder structure of AegisAI

The stack is listed in Table 5.1, and the default model name is qwen3:8b. Configuration is supplied through environment variables (DATABASE_URL, OLLAMA_BASE_URL,

OLLAMA_MODEL, BACKEND_URL and FRONTEND_URL), with a .env.example template in the repository.

Following the repository README, the backend is started with uvicorn on port 8000, the dashboard with npm install and npm run dev (served at http://localhost:5173), and the complete system with docker-compose up --build. The automated tests use a deterministic mock model provider behind the same interface as the Ollama provider, so they do not need the model to be running.

5.3 Implementation of the Policy Enforcement Gateway

The gateway is the central decision broker and is implemented with FastAPI. Agent A is prompted to work in proposal mode and to return a strictly typed JSON object containing response_text and an optional proposed_action with a tool_name and arguments. The gateway validates this object with Pydantic models, loads the conversation context from the PostgreSQL context store, such as earlier refunds for the same order, and runs the compiled policies held in memory. Each applicable rule returns an outcome, and the engine takes the verdict of highest priority, ESCALATE over BLOCK over MODIFY over ALLOW, so a high-severity violation always overrides a lower one.

The policy evaluation logic is in backend/app/policies/evaluator.py and the evaluation endpoint in backend/app/api/evaluate.py. If an unhandled exception occurs during evaluation, the catch block returns a default BLOCK verdict, so an action that could not be checked is never executed. During testing, the duplicate-refund policy DUPLICATE_REFUND_001 was hardened to ignore null values, so that a missing order ID cannot produce a false duplicate-refund match.



Code

Blame

611 lines (555 loc) · 26.6 KB



Raw



```

1  import re
2  from typing import Any, Dict, Optional
3  from app.schemas.chat import PolicyDecision, ProposedAction
4
5  def evaluate_high_risk_escalation(
6      customer_message: str,
7      agent_a_response: str,
8      policy_def: Dict[str, Any],
9  ) -> Optional[PolicyDecision]:
10     """Policy: HIGH_RISK_ESCALATION_001
11     Checks for fraud, legal threats, and severe escalation triggers.
12     """
13     policy_id = policy_def.get("policy_id", "HIGH_RISK_ESCALATION_001")
14     version = policy_def.get("version", "v1.0")
15     severity = policy_def.get("severity", "CRITICAL")
16     keywords = policy_def.get("parameters", {}).get("escalation_keywords", [
17         "fraud", "scam", "lawyer", "legal", "court", "sue", "police", "consumer court", "chargeback threat", "au
18     ])
19
20     combined_text = f"{customer_message} {agent_a_response}".lower()
21     matched_keywords = [kw for kw in keywords if re.search(r'\b' + re.escape(kw) + r'\b', combined_text)]
22
23     if matched_keywords:
24         return PolicyDecision(
25             decision="ESCALATE",

```

Figure 5.2: Policy evaluation and decision aggregation code

5.4 Implementation of Policy Definitions

Policies are stored as YAML so that business rules stay separate from gateway code, and the file is parsed at startup into strongly typed Pydantic models. Each policy has an ID, version, severity, target tool, conditions, an outcome and a remediation code. Listing 5.1 shows REFUND_LIMIT_001, which blocks a refund above ₹500 when no manager approval is present. The policies implemented are listed in Table 1.2. Policy versions are kept in a separate policy_versions table, and a change made through the API creates a new version instead of overwriting the old one.

```

policy_id: "REFUND_LIMIT_001"

version: "v1.0"

description: "Blocks unapproved refunds over ₹500"

severity: "HIGH"

target_tool: "issue_refund"

conditions:

  - field: "proposed_action.arguments.amount"

    operator: "GREATER_THAN"

    value: 500

  - field: "context.manager_approved"

    operator: "EQUALS"

    value: false

outcome: "BLOCK"

remediation_code: "REFUND_EXCEEDS_THRESHOLD"

```

Listing 5.1: YAML definition of REFUND_LIMIT_001

5.5 Implementation of the Tool Interceptor and Authorization Envelope

For an ALLOW decision the gateway creates an authorization envelope that names the permitted tool, its exact arguments and the decision ID. Agent B is a restricted operations agent that receives only this envelope and forwards it, with its function call, to the Tool Interceptor. The interceptor is a Python function that checks the structure, the types and the identity of every parameter, and it executes the mock tool only if the called tool and the called arguments both equal those in the envelope. No type coercion is allowed, so the string "400" does not match the integer 400. Any mismatch raises an exception and the tool does not run; for example, if the refund amount is changed from 400 to 3,000, the call is stopped. Agent A's code has no direct execution path to the mock tools module, so the interceptor is the only route to a tool.

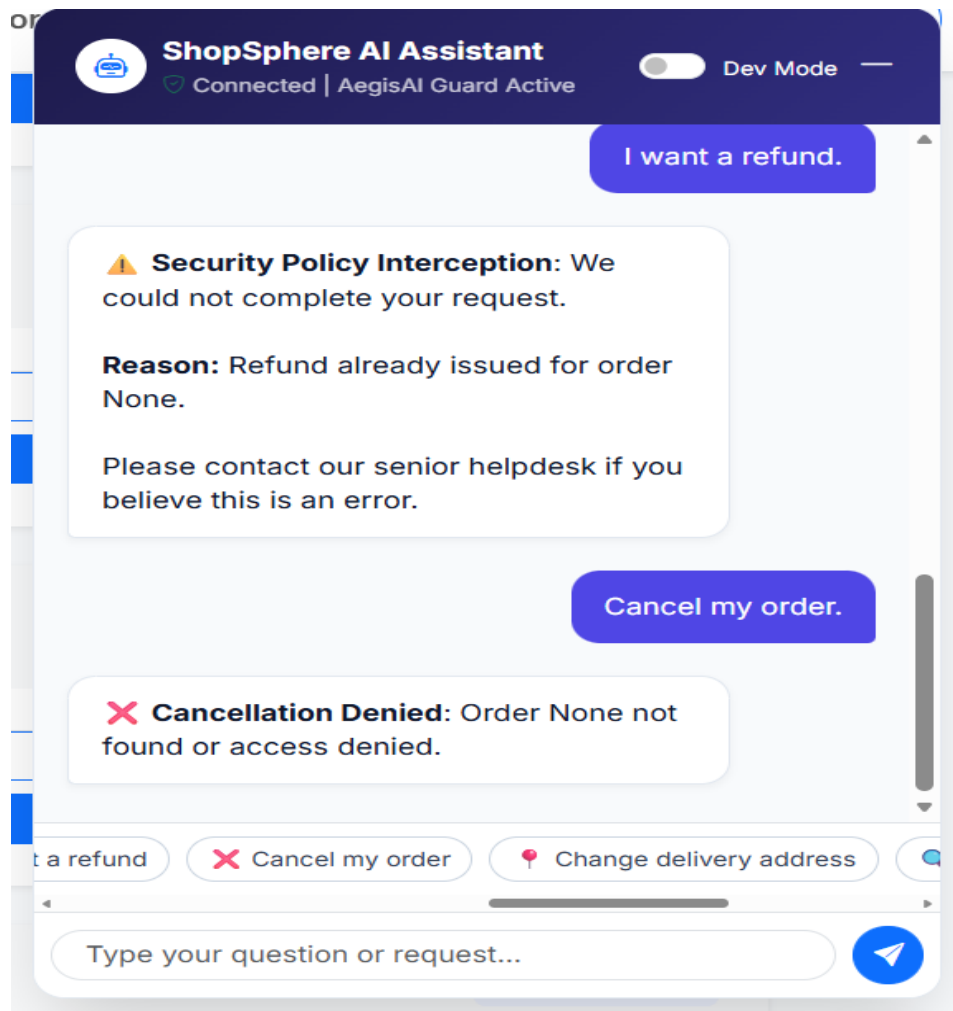


Figure 5.3: Tool Interceptor halting a call with altered arguments

5.6 Implementation of Audit Logging and Incident Handling

Every decision creates an audit event in PostgreSQL. The database has seven tables: conversations, messages, policies, policy_versions, audit_events, incidents and approvals, accessed through SQLAlchemy 2.x with Alembic migrations. An audit event records the request ID, conversation ID, customer ID, Agent A's response, the proposed action, the policy ID and version, the decision, severity, evidence, whether a tool was executed, the safe response and a timestamp. For BLOCK and ESCALATE decisions an incident record is also created, with the conversation and policy details. A manager can approve or reject a held action through the approval endpoints. The audit trail, incidents, policies and approvals are available through the REST endpoints `/api/audits`, `/api/incidents`, `/api/policies` and `/api/approvals`.

5.7 Implementation of the Dashboard

The dashboard is built with React 18, TypeScript and Vite and communicates with the backend through the REST API. The project documentation lists four screens: a Conversation Simulator for sending test messages through the pipeline, Audit History for

browsing recorded decisions, Incident Review for the incident queue, and Policy Management for viewing and updating policies. Recharts is used for charts.

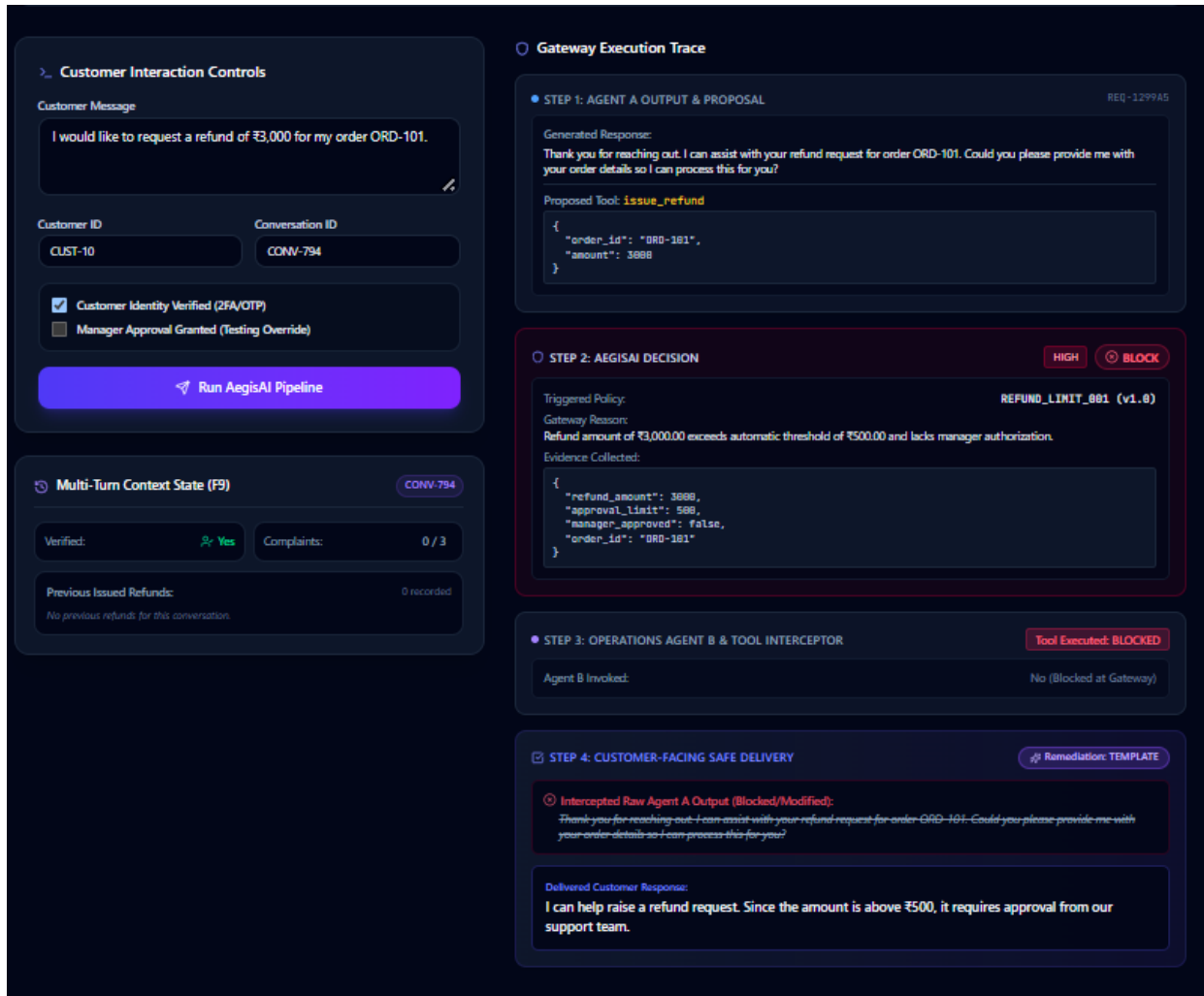


Figure 5.4: Conversation Simulator screen

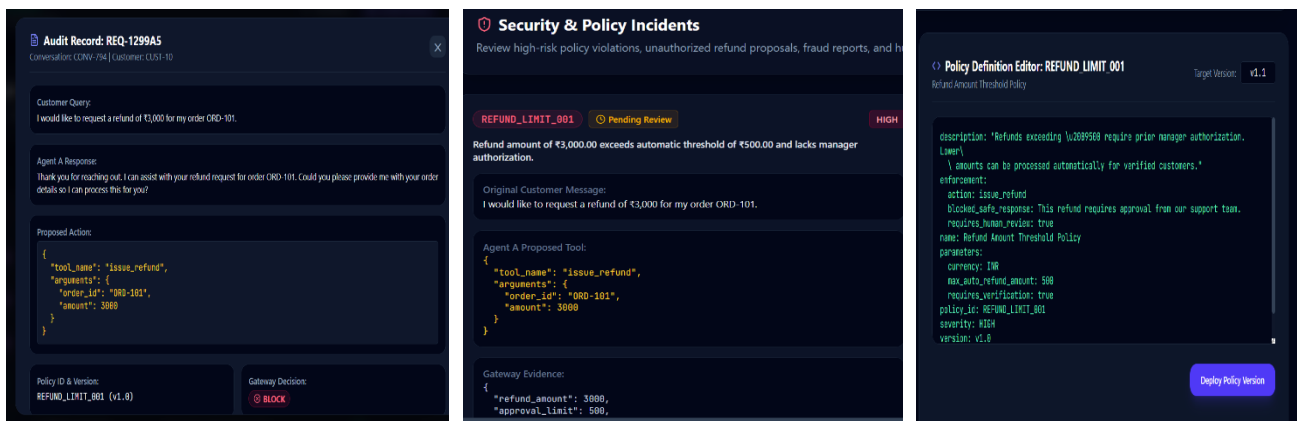


Figure 5.5: Audit History, Incident Review and Policy Management screens

5.8 Testing

Testing combined unit, workflow, integration and benchmark suites, and Table 5.2 lists the results recorded in the project's test walkthrough. The demonstration scenarios in the repository README exercise a ₹3,000 refund (BLOCK), a ₹400 refund (ALLOW),

unverified access to order details (BLOCK), an unverified delivery-date promise (MODIFY), a fraud or legal threat (ESCALATE) and altered tool arguments (BLOCK).

Per-Category Accuracy Matrix						
Policy Category	Cases	Correct	Accuracy	False Allows	False Blocks	Mean Latency
delivery-date	8	8	100.0%	0	0	2,120.69 ms
escalation	8	8	100.0%	0	0	2,142.14 ms
identity-verification	8	8	100.0%	0	0	2,111.96 ms
pii-protection	8	8	100.0%	0	0	2,123.84 ms
refund-limit	8	8	100.0%	0	0	2,141.19 ms
TOTAL / OVERALL	40	40	100.0%	0	0	2,127.96 ms

Automated Test Summary	
•	pytest tests/test_chatbot_accuracy.py: 6/6 PASSED
•	pytest tests/test_eval_harness.py: 40/40 PASSED
•	pytest tests/test_aegisai_integration.py: 11/11 PASSED
•	pytest in PolicySentinel/backend: 37/37 PASSED

Figure 5.6: Automated test results

Table 5.1– Test suites

TEST SUITE	CASES
Chatbot accuracy and context (tests/test_chatbot_accuracy.py)	6
40-case real benchmark harness (tests/test_eval_harness.py)	40
End-to-end interception suite (tests/test_aegisai_integration.py)	11
AegisAI gateway unit and workflow suite (run in PolicySentinel/backend)	37

5.8.1 Measured Results

All 40 benchmark cases produced the expected decision.

Table 5.2 – Decision accuracy

MEASURE	RESULT
Overall decision accuracy	100.0% (40/40)
False-allow rate	0.0% (0/40)
False-block rate	0.0% (0/40)
Argument invariant block rate	97.5%

Table 5.3 – Latency

LATENCY MEASURE	VALUE
Mean baseline latency (Agent A draft only)	4.44 ms
Mean latency with AegisAI gateway	2,127.96 ms
Median (P50) gateway latency	2,126.25 ms
90th percentile (P90)	2,163.57 ms
95th percentile (P95)	2,189.76 ms
Gateway interception overhead	+2,123.53 ms

The benchmark holds eight cases for each of five policy categories, and Table 5.3 gives the results, which were written to eval_results.csv.

Table 5.4: Benchmark results by policy category (N = 40)

POLICY CATEGORY	CASES	CORRECT	ACCURACY	FALSE ALLOWS	FALSE BLOCKS	MEAN LATENCY
Delivery-Date	8	8	100.0%	0	0	2,120.69 ms
Escalation	8	8	100.0%	0	0	2,142.14 ms
Identity-Verification	8	8	100.0%	0	0	2,111.96 ms
Pii-Protection	8	8	100.0%	0	0	2,123.84 ms
Refund-Limit	8	8	100.0%	0	0	2,141.19 ms
Total	40	40	100.0%	0	0	2,127.96 ms

The gateway returned the expected decision in all 40 cases (100% accuracy), with no false allows and no false blocks.

5.8.2 Analysis of Results

- **Accuracy.** Every one of the 40 cases was decided correctly, with no false allows and no false blocks. The absence of false allows is the more important result for a safety gateway, because a false allow means an unsafe proposal went through.
- **Consistency across categories.** All five categories reached 100% accuracy, and their mean latencies lie close together (about 2,112 ms to 2,142 ms), so no single policy category is noticeably slower than the others.

- **Latency cost.** The gateway adds about 2,123.53 ms on average to each interaction compared with the Agent A draft alone. The gap between the median (2,126.25 ms) and the P95 (2,189.76 ms) is small, so the delay is steady and not dominated by occasional slow cases.
- **Argument invariant block rate.** The argument invariant block rate was 97.5%, which is below the 100% decision accuracy.
- **Scope of the benchmark.** The benchmark covers 40 cases in five categories. Per the project disclaimer, the results apply to the defined ShopSphere environment, the configured policies, the supported tools and the test scenarios, and do not guarantee detection or prevention of all violations.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

This project addressed a gap in AI-powered customer support: a chatbot’s response or action can break company policy, and in most deployments nothing independent checks it before it reaches the customer or a business tool. AegisAI was designed and implemented as a pre-execution policy enforcement gateway for this problem. It separates proposal from execution, so that Agent A can only propose a reply and an optional action, and the restricted Agent B can carry out only what the gateway has authorized. A deterministic policy engine evaluates each proposal against YAML-defined business policies and returns ALLOW, MODIFY, BLOCK or ESCALATE, using the priority ESCALATE over BLOCK over MODIFY over ALLOW. For approved actions, the Tool Interceptor compares the call with the authorization envelope using exact equality on the tool and every argument. A Remediation Agent produces a compliant replacement reply, and every decision is recorded in a PostgreSQL audit store, with incident records for BLOCK and ESCALATE decisions. The system was built with Python, FastAPI, LangGraph, Qwen3-8B served by Ollama, PostgreSQL and a React dashboard.

The system was evaluated in a mock e-commerce environment. On a benchmark of 40 cases, eight in each of five policy categories (delivery date, escalation, identity verification, PII protection and refund limit), the gateway returned the expected decision in all 40 cases, with no false allows and no false blocks. The gateway test suite (37 tests) and the end-to-end interception suite (11 tests) also passed. The cost was an added latency of about 2.1 s per request: the mean was 2,127.96 ms through the gateway, against 4.44 ms for the Agent A draft stage alone.

These results show that externalised, deterministic policies can give predictable and auditable control over an AI customer-support agent, because the decision does not depend on the language model’s own behaviour. The results have limits. The benchmark is small, and the system was tuned against failures found during development. The evaluation used a mock store and a single locally hosted model, the latency cost matters for real-time use, and robustness to adversarial prompts was not measured. A larger independently labelled evaluation is therefore needed before the accuracy can be generalised.

6.2 Future Work

The following directions follow from the limitations above and from the project documentation:

- **Larger and broader evaluation.** Extend the benchmark with more cases per category, labelled independently of the developers, together with adversarial prompts, and repeat the evaluation beyond the mock store and a single locally hosted model.
- **Latency reduction.** Measure how the added 2.1 s per request is divided between the stages of the pipeline, and reduce it.
- **Dynamic policy synthesis.** Generate policies from natural-language policy documents using autoformalization, an approach explored for agent policies by Mondl et al. [13], to reduce the manual effort of writing YAML policies.
- **Cryptographic signing of authorization envelopes.** Add token signing so that the payload passed between agents can be verified.
- **Scalability.** Assess the gateway in distributed, multi-tenant e-commerce deployments.
- **Human approval and policy testing.** Pause and resume workflows for manager approval of high-risk actions using LangGraph interrupts, and add a shadow mode that logs policy violations without blocking, so that new policies can be tested before they are enforced. Both are listed as planned in the project documentation.
- **Operational tooling.** Add a metrics dashboard, richer dashboard analytics and a replay mode for past decisions, which are listed in the repository roadmap.

REFERENCES

- [1] S. Gupta, B. Arief, and R. de Lemos, “Agentic AI vs non-agentic AI: Motivation, security implications, and research foundations,” presented at the Workshop on AI Safety and Security (AI-SS 2026), Canterbury, U.K., Apr. 2026.
- [2] *Moffatt v. Air Canada*, 2024 BCCRT 149, Civil Resolution Tribunal of British Columbia, Feb. 2024.
- [3] AI Incident Database, “Chevrolet dealer chatbot agrees to sell Tahoe for \$1,” Dec. 18, 2023. [Online]. Available: <https://incidentdatabase.ai/entities/chevrolet-of-watsonville>
- [4] Associated Press, “NYC’s AI chatbot was caught telling businesses to break the law. The city isn’t taking it down,” Apr. 2024. [Online]. Available: <https://ktla.com/news/technology/ap-technology/ap-nycs-ai-chatbot-was-caught-telling-businesses-to-break-the-law-the-city-isnt-taking-it-down/amp/>
- [5] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen, “NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails,” NVIDIA Research, Oct. 2023. [Online]. Available: https://research.nvidia.com/publication/2023-10_nemo-guardrails-toolkit-controllable-and-safe-llm-applications-programmable
- [6] H. Inan et al., “Llama Guard: LLM-based input-output safeguard for human-AI conversations,” arXiv:2312.06674, Dec. 2023.
- [7] Guardrails AI, “Guardrails,” GitHub repository. [Online]. Available: <https://github.com/guardrails-ai/guardrails>
- [8] Amazon Web Services, “Detect and filter harmful content by using Amazon Bedrock Guardrails,” Amazon Bedrock User Guide. [Online]. Available: <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails.html>
- [9] H. Wang, C. M. Poskitt, and J. Sun, “AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents,” in Proc. IEEE/ACM 48th Int. Conf. Softw. Eng. (ICSE), Rio de Janeiro, Brazil, Apr. 2026; arXiv:2503.18666.
- [10] Y. Li et al., “VIGIL: Runtime enforcement of behavioral specifications in AI agent skills,” arXiv:2606.26524, Jun. 2026.
- [11] V. P. Bhardwaj, “Agent behavioral contracts: Formal specification and runtime enforcement for reliable autonomous AI agents,” arXiv:2602.22302, Feb. 2026.
- [12] A. Sigdel and R. Baral, “Guardrails as infrastructure: Policy-first control for tool-orchestrated workflows,” arXiv:2603.18059, Mar. 2026.
- [13] A. Mondl, M. Maisel, and J. H. Brock, “Autoformalization of agent instructions into policy-as-code,” arXiv:2606.26649, Jun. 2026.
- [14] S. Gaurav, J. Heikkonen, and J. Chaudhary, “Governance-as-a-Service: A multi-agent framework for AI system compliance and policy enforcement,” arXiv:2508.18765, Aug. 2025.
- [15] V. Tran, T. Sharma, T. S. Dhesi, and N. Feamster, “Rethinking agent security as a networking problem,” arXiv:2608.12172, Aug. 2026.
- [16] K. Tallam, “Authorization propagation in multi-agent AI systems: Identity governance as infrastructure,” arXiv:2605.05440, May 2026.
- [17] I. Habler, K. Huang, V. S. Narajala, and P. Kulkarni, “Building a secure agentic AI application leveraging A2A protocol,” arXiv:2504.16902, Apr. 2025.
- [18] “Qwen 3 8B,” BestLLMFor model catalog. [Online]. Available: <https://bestllmfor.com/catalog/qwen3-8b/>
- [19] “Using Ollama models,” Presenton documentation. [Online]. Available: <https://docs.presenton.ai/configurations/using-ollama-models.md>